

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ
ПЕДАГОГИЧЕСКИЙ УНИВЕРСИТЕТ им. А. И. ГЕРЦЕНА»

**Институт информационных технологий и технологического образования
кафедра информационных технологий и электронного обучения**

Основная профессиональная образовательная программа
Направление подготовки 09.03.01 Информатика и вычислительная техника
Направленность (профиль) «Технологии разработки программного обеспечения
и обработки больших данных»
форма обучения – очная

Курсовая работа

по дисциплине «Прикладные информационные технологии»
Разработка и контейнеризация веб-приложения для управления заметками
(NoteKeeper) с использованием Docker Compose, reverse-proxy на базе NGINX и
автоматическим получением HTTPS-сертификата через Let's Encrypt

Обучающейся 3 курса
Нуриева Джамиля Эльхановна

Руководитель:
ст. преп. Аксютин П. А.

Санкт-Петербург
2026

ОГЛАВЛЕНИЕ

ВВЕДЕНИЕ	3
Глава 1. Теоретические основы технологий контейнеризации и веб-безопасности	6
1.1 Платформа Docker	6
1.2 Инструмент Docker Compose	7
1.3 Веб-сервер Nginx	8
1.4 Система Let's Encrypt	8
1.5 Обзор и сравнение технологий	10
Глава 2. Практическая реализация веб-приложения NoteKeeper	13
2.1 Общая архитектура приложения	13
2.3 Реализация серверной части (бэкенд)	14
2.4 Реализация клиентской части (фронтенд)	16
2.5 Контейнеризация приложения	19
2.6 Оркестрация с Docker Compose	20
2.7 Настройка HTTPS с Let's Encrypt	21
2.8 Развёртывание на сервере	22
2.9 Тестирование приложения	23
ЗАКЛЮЧЕНИЕ	25
СПИСОК ЛИТЕРАТУРЫ	27

ВВЕДЕНИЕ

В современной веб-разработке всё большее распространение получают технологии контейнеризации, позволяющие упаковывать приложения со всеми зависимостями в изолированные среды — контейнеры. Данный подход обеспечивает переносимость приложений между различными средами выполнения, упрощает процесс развертывания и масштабирования, а также повышает воспроизводимость окружения. Особую актуальность приобретает автоматизация получения SSL-сертификатов для обеспечения безопасного HTTPS-соединения, так как вопросы кибербезопасности и защиты данных пользователей становятся критическими для любого веб-проекта. Инструменты Docker Compose, NGINX и Let's Encrypt образуют современный технологический стек, позволяющий эффективно решать задачи контейнеризации и обеспечения безопасности веб-приложений.

Объектом исследования выступают технологии контейнеризации и автоматизации развертывания веб-приложений.

Предмет исследования — процесс разработки, контейнеризации и развертывания веб-приложения с поддержкой HTTPS.

Целью работы является разработка веб-приложения для управления заметками, контейнеризирование его с использованием Docker, настройка reverse-proxy на базе NGINX и организация автоматического получения и обновления SSL-сертификатов Let's Encrypt.

Для достижения поставленной цели необходимо решить следующие задачи:

1. Изучить теоретические основы платформы Docker, инструмента оркестрации Docker Compose, веб-сервера NGINX и системы автоматической выдачи сертификатов Let's Encrypt.
2. Провести сравнительный анализ альтернативных технологий для каждого компонента системы и обосновать выбор используемых решений.
3. Разработать веб-приложение «NoteKeeper», включающее серверную часть с REST API на базе Express.js и клиентскую часть с веб-интерфейсом на нативном JavaScript.
4. Создать Dockerfile для контейнеризации бэкенда и фронтенда, а также настроить конфигурационные файлы для проксирования API-запросов.
5. Написать файл docker-compose.yml для оркестрации всех сервисов приложения.
6. Настроить reverse-проxy на базе NGINX и организовать автоматическое получение SSL-сертификата от Let's Encrypt.
7. Развернуть приложение на сервере и провести его тестирование, включая проверку работоспособности HTTPS-соединения.
8. Составить документацию по запуску проекта в формате README.md, оформленную по образцу документации на Docker Hub.

В работе использовались следующие методы исследования: анализ научно-технической и справочной литературы по теме контейнеризации и веб-безопасности; сравнительный анализ технологий; проектирование архитектуры приложения; программная реализация с использованием языков JavaScript и SQL; эксперимент

по развертыванию приложения на сервере и тестированию его работы.

Работа будет состоять из теоретической части, в которой будут подробно изучаться технологии контейнеризации Docker, инструмент оркестрации Docker Compose, веб-сервер NGINX в роли reverse-proxy и система автоматической выдачи SSL-сертификатов Let's Encrypt. Будет рассмотрена архитектура каждого компонента, их ключевые особенности и принципы взаимодействия.

В практической части, будет выполнена разработка веб-приложения «NoteKeeper» для управления заметками. Будет реализован программный код на JavaScript с использованием фреймворка Express.js для серверной части и нативного HTML/CSS/JS для клиентской части. Будут составлены конфигурационные файлы Dockerfile для контейнеризации бэкенда и фронтенда, а также файл docker-compose.yml для оркестрации всех сервисов. Будет настроен reverse-proxy на базе NGINX и организовано автоматическое получение SSL-сертификата Let's Encrypt с помощью инструмента Certbot.

Глава 1. Теоретические основы технологий контейнеризации и веб-безопасности

1.1 Платформа Docker

Docker представляет собой программную платформу для разработки, доставки и запуска приложений в контейнерах. Платформа обеспечивает создание контейнеров, автоматизацию их запуска и развертывания, а также управление жизненным циклом. Ключевой особенностью Docker является возможность одновременного выполнения множества контейнеров на одной хост-машине при минимальных накладных расходах.

Платформа Docker распространяется в двух редакциях: бесплатная Community Edition (CE) под лицензией Apache 2.0 и платная Enterprise Edition (EE), предназначенная для коммерческого использования и распространяемая по проприетарной лицензии. Изначально система была разработана для операционных систем семейства Linux и UNIX-подобных систем, однако начиная с 2015 года в программное обеспечение была добавлена поддержка платформы Windows, что значительно расширило область применения инструмента.

Контейнеризация как технологический подход

Контейнеризация представляет собой метод упаковки программного обеспечения со всеми его зависимостями в изолированную среду исполнения, называемую контейнером. Принципиальное отличие контейнера от виртуальной машины заключается в том, что виртуальная машина эмулирует полноценный компьютер с отдельной операционной системой, тогда как контейнер использует ядро хост-системы и изолирует только процессы приложения.

Контейнер включает в себя следующие компоненты:

- исполняемый код приложения;
- системные библиотеки;
- переменные окружения;
- конфигурационные файлы.

Ключевые особенности контейнеров делают их удобным инструментом для современной разработки программного обеспечения. Во-первых, контейнеры являются легковесными — они занимают от десятка до сотен мегабайт, в то время как виртуальная машина требует гигабайты дискового пространства. Во-вторых, контейнеры изолированы друг от друга: падение одного контейнера не затрагивает работу остальных. В-третьих, контейнеры обладают переносимостью — один и тот же образ запустится одинаково на ноутбуке разработчика, тестовом сервере и в производственном окружении[1].

1.2 Инструмент Docker Compose

Docker Compose представляет собой инструмент для определения и запуска многоконтейнерных приложений на платформе Docker. В отличие от ручного управления контейнерами посредством команд `'docker run'`, Compose позволяет декларативно описать всю архитектуру приложения в одном YAML-файле и запустить все компоненты единой командой.

Основная проблема, которую решает Docker Compose, заключается в следующем: при разработке веб-приложения часто требуется одновременно запустить несколько сервисов — веб-сервер, базу данных, кэш, очередь сообщений. Ручной запуск каждого контейнера с правильными параметрами сетей, томов и

переменных окружения представляет собой трудоёмкий и подверженный ошибкам процесс. Docker Compose полностью автоматизирует данную задачу, обеспечивая воспроизводимость окружения и упрощая управление многоконтейнерными приложениями.

1.3 Веб-сервер Nginx

Nginx представляет собой веб-сервер с открытым исходным кодом, разработанный российским программистом Игорем Сысоевым. В отличие от традиционных серверов, использующих потоковую или процессную модель обработки запросов, Nginx использует асинхронную, событийно-ориентированную архитектуру, которая делает его исключительно эффективным при обработке множества одновременных соединений.

Основной функцией Nginx является веб-серверная обработка HTTP-запросов, однако он также может функционировать как обратный прокси-сервер, балансировщик нагрузки, кэш-сервер и даже почтовый прокси-сервер. Nginx предлагает впечатляющий набор функций, которые делают его незаменимым инструментом для современных веб-проектов любого масштаба — от базовой раздачи статического контента до сложных сценариев обработки запросов[2].

1.4 Система Let's Encrypt

Let's Encrypt представляет собой автоматизированный центр сертификации (Certificate Authority, CA), запущенный в 2016 году организацией Internet Security Research Group (ISRG). Основная цель проекта — сделать шифрование повсеместным и бесплатным[3]. За

годы работы система выдала более трёх миллиардов сертификатов, покрывая около 90% всех HTTPS-сайтов в сети Интернет.

Ключевые принципы Let's Encrypt

Система Let's Encrypt базируется на следующих основополагающих принципах:

- Бесплатность — отсутствие скрытых платежей и премиум-функций, полная доступность для всех желающих;
- Автоматизация — полный жизненный цикл сертификата управляется через программный интерфейс (API);
- Безопасность — использование современных криптографических стандартов и практик;
- Прозрачность — публичное логирование всех выданных сертификатов;
- Открытость — протокол ACME является открытым стандартом, поддерживаемым различными реализациями.

Протокол ACME

Automatic Certificate Management Environment (ACME) — это протокол, который позволяет автоматически получать, устанавливать и обновлять SSL/TLS-сертификаты без участия человека[4]. Процесс получения сертификата включает следующие этапы:

1. Запрос сертификата — клиент отправляет запрос на получение сертификата для указанного домена;
2. Валидация домена — центр сертификации Let's Encrypt проверяет, что заявитель действительно контролирует указанный домен;
3. Выдача сертификата — после успешной валидации центр сертификации выдаёт запрошенный сертификат;

4. Автообновление — за 30 дней до истечения срока действия сертификат обновляется автоматически.

Практическое применение

Система Let's Encrypt кардинально изменила ландшафт веб-безопасности, сделав SSL-сертификаты доступными для всех. Сегодня отсутствие HTTPS является скорее исключением, чем правилом.

Рекомендуемые сценарии использования Let's Encrypt:

- все публичные веб-сайты и API;
- тестовые (dev/staging) окружения;
- личные проекты и стартапы;
- микросервисная архитектура.

Сценарии, в которых целесообразно рассмотреть платные альтернативы:

- корпоративные сайты, для которых важен Extended Validation (EV), отображающий название компании в адресной строке;
- приложения с особыми требованиями к соглашению об уровне обслуживания (SLA);
- случаи, когда требуется круглосуточная техническая поддержка.

1.5 Обзор и сравнение технологий

Сравнение решений для reverse-proxy и HTTPS

В ходе работы рассматривались четыре варианта организации reverse-proxy и HTTPS.

Первый вариант — связка nginx-proxy и acme-companion, которая была выбрана для реализации. Данная связка автоматически настраивает reverse-proxy на основе Docker-меток. При запуске контейнера с переменной VIRTUAL_HOST nginx-proxy создаёт конфигурацию для перенаправления запросов. acme-companion

отслеживает контейнеры с меткой `LETSencrypt_HOST` и автоматически получает сертификаты Let's Encrypt. Главное преимущество — минимальная конфигурация: достаточно добавить переменные окружения, не требуется писать сложные конфиги.

Второй вариант — Traefik. Это более современный reverse-проxy, имеющий встроенную поддержку Let's Encrypt. Однако его конфигурация сложнее: требуется описывать middleware, роутеры и сервисы в YAML или через Docker labels. Для учебного проекта это избыточно.

Третий вариант — Caddy. Он отличается автоматическим HTTPS без дополнительной настройки и имеет простой Caddyfile. Минус — менее удобная интеграция с Docker по сравнению с nginx-проxy.

Четвёртый вариант — ручная настройка Nginx и certbot. Он требует самостоятельного создания конфигурационных файлов Nginx и настройки cron-заданий для обновления сертификатов. Даёт полный контроль, но трудоёмок и подвержен ошибкам.

В работе был выбран nginx-проxy и acme-companion, так как эта связка совмещает в себе минимальную конфигурацию, широкое распространение в учебных проектах и лёгкую интеграцию с Docker Compose через переменные окружения.

Сравнение систем управления базами данных

В ходе работы рассматривались четыре системы управления базами данных.

Первый вариант — PostgreSQL, который был выбран для реализации. Это объектно-реляционная СУБД с открытым исходным кодом, обеспечивающая полную поддержку ACID-транзакций, высокое соответствие стандарту SQL и развитую систему типов

данных. PostgreSQL имеет официальный Docker-образ с удобной настройкой через переменные окружения.

Второй вариант — MySQL. Это наиболее популярная реляционная СУБД в веб-разработке, однако она уступает PostgreSQL в соответствии стандарту SQL.

Третий вариант — SQLite. Это встраиваемая база данных, не требующая отдельного контейнера. Она не подходит для production-сценариев из-за отсутствия конкурентного доступа к данным.

Четвёртый вариант — MongoDB. Это документо-ориентированная NoSQL-база данных, которая избыточна для структурированных данных заметок, имеющих чёткую схему.

В работе был выбран PostgreSQL, так как он обеспечивает надёжность, полную поддержку ACID-транзакций, соответствие стандарту SQL и хорошую интеграцию с Docker.

Сравнение бэкенд-фреймворков

В ходе работы рассматривались четыре бэкенд-фреймворка.

Первый вариант — Express.js, который был выбран для реализации. Это минималистичный фреймворк для Node.js, использующий асинхронную событийно-ориентированную модель. Он имеет огромную экосистему и низкий порог входа. Размер образа на основе node:18-alpine составляет около 150 мегабайт.

Второй вариант — Flask. Это микрофреймворк для Python, который синхронен по умолчанию, что снижает производительность при операциях ввода-вывода.

Третий вариант — FastAPI. Это современный асинхронный фреймворк для Python, который сложнее для начинающих из-за необходимости использования тайп-хинтов.

Четвёртый вариант — Gin. Это высокопроизводительный фреймворк для языка Go, требующий знания самого языка Go.

Для работы был выбран Express.js, так как он обладает низким порогом входа, лёгковесностью образа и большим количеством готовых примеров реализации CRUD-операций.

Сравнение решений для фронтенда

В ходе работы рассматривались два варианта реализации фронтенда.

Первый вариант — статический Nginx с нативным JavaScript, который был выбран для реализации. Размер образа на основе nginx:alpine составляет около 5 мегабайт. Такое решение не требует этапа сборки.

Второй вариант — React или Vue.js. Эти фреймворки требуют этапа сборки и значительно увеличивают размер образа — до 100 мегабайт и более.

В работе был выбран статический Nginx с нативным JavaScript, так как это решение обеспечивает минимальный размер образа, отсутствие этапа сборки и достаточную функциональность для учебного проекта.

Глава 2. Практическая реализация веб-приложения NoteKeeper

2.1 Общая архитектура приложения

Разрабатываемое приложение NoteKeeper представляет собой веб-сервис для создания и управления заметками. Архитектура включает следующие компоненты:

Компонент	Технология	Назначение
Клиентская	HTML, CSS,	Пользовательский интерфейс

Компонент	Технология	Назначение
часть	JavaScript	
Серверная часть	Node.js + Express.js	REST API, бизнес-логика
База данных	PostgreSQL	Хранение заметок
Reverse-proxy	NGINX	Маршрутизация трафика, HTTPS

Схема взаимодействия компонентов:

Браузер → NGINX (порты 80/443) → Frontend (статический Nginx)
→ Backend (Express.js) → PostgreSQL

2.3 Реализация серверной части (бэкенд)

Для реализации серверной части был выбран фреймворк Express.js. Приложение предоставляет REST API со следующими эндпоинтами:

HTTP метод	URL	Описание
GET	/notes	Получение всех заметок
GET	/notes/:id	Получение одной заметки
POST	/notes	Создание заметки
PUT	/notes/:id	Обновление заметки
DELETE	/notes/:id	Удаление заметки

Листинг 2.1 — backend/server.js

```
const express = require('express');
const { Pool } = require('pg');
```

```

const cors = require('cors');

const app = express();
const port = 3000;
// Middleware
app.use(cors());
app.use(express.json());
// Подключение к PostgreSQL
const pool = new Pool({
  host: process.env.DB_HOST || 'database',
  port: 5432,
  user: process.env.DB_USER || 'postgres',
  password: process.env.DB_PASSWORD || 'postgres',
  database: process.env.DB_NAME || 'notesdb'
});
// Создание таблицы при запуске
pool.query(`
  CREATE TABLE IF NOT EXISTS notes (
    id SERIAL PRIMARY KEY,
    title VARCHAR(200) NOT NULL,
    content TEXT,
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
  )
`);
.catch(err => console.error('Ошибка создания таблицы:', err));
// GET /notes - получить все заметки
app.get('/notes', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM notes ORDER BY id DESC');
    res.json(result.rows);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
// GET /notes/:id - получить одну заметку
app.get('/notes/:id', async (req, res) => {
  try {
    const result = await pool.query('SELECT * FROM notes WHERE id = $1', [req.params.id]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Заметка не найдена' });
    }
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
// POST /notes - создать заметку
app.post('/notes', async (req, res) => {
  const { title, content } = req.body;
  if (!title) {
    return res.status(400).json({ error: 'Заголовок обязателен' });
  }
  try {
    const result = await pool.query(
      'INSERT INTO notes (title, content) VALUES ($1, $2) RETURNING *',
      [title, content || '']
    );
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

```

```

// PUT /notes/:id - обновить заметку
app.put('/notes/:id', async (req, res) => {
  const { title, content } = req.body;
  try {
    const result = await pool.query(
      'UPDATE notes SET title = $1, content = $2 WHERE id = $3 RETURNING *',
      [title, content, req.params.id]
    );
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Заметка не найдена' });
    }
    res.json(result.rows[0]);
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// DELETE /notes/:id - удалить заметку
app.delete('/notes/:id', async (req, res) => {
  try {
    const result = await pool.query('DELETE FROM notes WHERE id = $1 RETURNING *',
    [req.params.id]);
    if (result.rows.length === 0) {
      return res.status(404).json({ error: 'Заметка не найдена' });
    }
    res.json({ message: 'Заметка удалена' });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});

// Запуск сервера
app.listen(port, () => {
  console.log(`Backend запущен на порту ${port}`);
});

```

Листинг 2.2 — backend/package.json

```

{
  "name": "note-app-backend",
  "version": "1.0.0",
  "description": "REST API для заметок",
  "main": "server.js",
  "scripts": {
    "start": "node server.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0",
    "cors": "^2.8.5"
  }
}

```

2.4 Реализация клиентской части (фронтенд)

Фронтенд реализован в виде одностраничного приложения на нативном JavaScript.

Листинг 2.3 — frontend/index.html

```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>NoteKeeper - Мои заметки</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <div class="container">
    <h1> 📝 NoteKeeper</h1>

    <!-- Форма создания/редактирования -->
    <div class="form-card">
      <h2 id="form-title">Новая заметка</h2>
      <input type="text" id="note-title" placeholder="Заголовок">
      <textarea id="note-content" rows="5" placeholder="Текст заметки..."></textarea>
      <input type="hidden" id="edit-id">
      <button id="save-btn" class="btn-primary">Сохранить</button>
      <button id="cancel-btn" class="btn-secondary" style="display:none">Отмена</button>
    </div>

    <!-- Список заметок -->
    <div id="notes-list" class="notes-list">
      <div class="loading">Загрузка заметок...</div>
    </div>
  </div>
  <script src="script.js"></script>
</body>
</html>
```

Листинг 2.4 — frontend/script.js

```
const API_URL = '/api'; // Проксируется через Nginx

let notes = [];
// Загрузка заметок
async function loadNotes() {
  try {
    const response = await fetch(`${API_URL}/notes`);
    notes = await response.json();
    renderNotes();
  } catch (error) {
    console.error('Ошибка загрузки:', error);
    document.getElementById('notes-list').innerHTML = '<div class="loading">Ошибка
загрузки заметок</div>';
  }
}

// Отображение заметок
function renderNotes() {
  const container = document.getElementById('notes-list');
  if (notes.length === 0) {
    container.innerHTML = '<div class="loading">Нет заметок. Создайте первую!</div>';
    return;
  }

  container.innerHTML = notes.map(note => `
```

```

    <div class="note-card" data-id="${note.id}">
      <h3>${escapeHtml(note.title)}</h3>
      <p>${escapeHtml(note.content || '').substring(0, 200)}</p>
      <div class="note-date">${new Date(note.created_at).toLocaleString('ru-RU')}</div>
      <div class="card-buttons">
        <button class="btn-edit" onclick="editNote(${note.id})">✎
Редактировать</button>
        <button class="btn-delete" onclick="deleteNote(${note.id})">🗑
Удалить</button>
      </div>
    </div>
  `).join('');
}
// Создание заметки
async function createNote(title, content) {
  const response = await fetch(`${API_URL}/notes`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, content })
  });
  if (response.ok) {
    await loadNotes();
    return true;
  }
  return false;
}
// Обновление заметки
async function updateNote(id, title, content) {
  const response = await fetch(`${API_URL}/notes/${id}`, {
    method: 'PUT',
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify({ title, content })
  });
  if (response.ok) {
    await loadNotes();
    return true;
  }
  return false;
}
// Удаление заметки
async function deleteNote(id) {
  if (confirm('Удалить заметку?')) {
    await fetch(`${API_URL}/notes/${id}`, { method: 'DELETE' });
    await loadNotes();
  }
}
// Редактирование (заполнение формы)
async function editNote(id) {
  const response = await fetch(`${API_URL}/notes/${id}`);
  const note = await response.json();

  document.getElementById('form-title').textContent = 'Редактирование заметки';
  document.getElementById('note-title').value = note.title;
  document.getElementById('note-content').value = note.content || '';
  document.getElementById('edit-id').value = note.id;
  document.getElementById('cancel-btn').style.display = 'inline-block';
}
// Сброс формы
function resetForm() {
  document.getElementById('form-title').textContent = 'Новая заметка';
  document.getElementById('note-title').value = '';
}

```

```

    document.getElementById('note-content').value = '';
    document.getElementById('edit-id').value = '';
    document.getElementById('cancel-btn').style.display = 'none';
}
// Обработчик сохранения
document.getElementById('save-btn').addEventListener('click', async () => {
    const title = document.getElementById('note-title').value.trim();
    const content = document.getElementById('note-content').value;
    const editId = document.getElementById('edit-id').value;

    if (!title) {
        alert('Введите заголовок');
        return;
    }

    if (editId) {
        await updateNote(editId, title, content);
        resetForm();
    } else {
        await createNote(title, content);
        resetForm();
    }
});
document.getElementById('cancel-btn').addEventListener('click', resetForm);
// Защита от XSS
function escapeHtml(str) {
    if (!str) return '';
    return str.replace(/&lt;&gt;/g, function(m) {
        if (m === '&') return '&amp;';
        if (m === '<') return '&lt;';
        if (m === '>') return '&gt;';
        return m;
    });
}
// Загрузка при старте
loadNotes();

```

2.5 Контейнеризация приложения

Листинг 2.5 — backend/Dockerfile

```

FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install --omit=dev
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]

```

Листинг 2.6 — frontend/Dockerfile

```

FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY . /usr/share/nginx/html
EXPOSE 80

```

Листинг 2.7 — frontend/nginx.conf

```

server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;

    # Проксирование запросов к API на бэкенд
    location /api/ {
        proxy_pass http://backend:3000/;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
    }
    # Статические файлы
    location / {
        try_files $uri $uri/ /index.html;
    }
}

```

2.6 Оркестрация с Docker Compose

Листинг 2.8 — docker-compose.yml

```

version: '3.8'

services:
    # База данных
    database:
        image: postgres:15-alpine
        environment:
            POSTGRES_USER: ${DB_USER:-postgres}
            POSTGRES_PASSWORD: ${DB_PASSWORD:-postgres}
            POSTGRES_DB: ${DB_NAME:-notesdb}
        volumes:
            - postgres_data:/var/lib/postgresql/data
        networks:
            - app-network
        restart: unless-stopped
    # Бэкенд API
    backend:
        build: ./backend
        environment:
            DB_HOST: database
            DB_USER: ${DB_USER:-postgres}
            DB_PASSWORD: ${DB_PASSWORD:-postgres}
            DB_NAME: ${DB_NAME:-notesdb}
        depends_on:
            - database
        networks:
            - app-network
        restart: unless-stopped
    # Фронтенд
    frontend:
        build: ./frontend
        networks:
            - app-network
        restart: unless-stopped
    # Reverse-proxy с автоматическим HTTPS

```

```

nginx-proxy:
  image: nginxproxy/nginx-proxy
  ports:
    - "80:80"
    - "443:443"
  volumes:
    - /var/run/docker.sock:/tmp/docker.sock:ro
    - ./certs:/etc/nginx/certs
    - ./vhost:/etc/nginx/vhost.d
  networks:
    - app-network
  restart: unless-stopped
# Автоматическое получение сертификатов Let's Encrypt
acme-companion:
  image: nginxproxy/acme-companion
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock:ro
    - ./certs:/etc/nginx/certs
    - ./vhost:/etc/nginx/vhost.d
  environment:
    - DEFAULT_EMAIL=${LETSENCRYPT_EMAIL}
  depends_on:
    - nginx-proxy
  networks:
    - app-network
  restart: unless-stopped
networks:
  app-network:
    driver: bridge
volumes:
  postgres_data:

```

2.7 Настройка HTTPS с Let's Encrypt

Для получения SSL-сертификата использовался инструмент Certbot — официальный клиент Let's Encrypt.

Процесс получения сертификата включает несколько шагов. Сначала необходимо временно остановить контейнер с NGINX, чтобы освободить порт 80 для валидации домена. Это делается командой `docker compose stop nginx`.

Затем запускается Certbot в режиме standalone командой `certbot certonly --standalone -d ваш-домен.ru --email ваша-почта@example.com --non-interactive --agree-tos`. Параметр `-d` указывает домен для сертификата, `--email` — адрес для уведомлений о продлении, флаги `--non-interactive` и `--agree-`

tos отключают интерактивный режим и подтверждают согласие с условиями Let's Encrypt[3].

После успешного получения сертификат сохраняется в `/etc/letsencrypt/live/ваш-домен.ru/` в виде файлов `fullchain.pem` (сертификат) и `privkey.pem` (закрытый ключ). Эти файлы необходимо скопировать в папку проекта. Для этого создаётся директория `certs` командой `mkdir -p certs`, после чего выполняется копирование: `cp /etc/letsencrypt/live/ваш-домен.ru/fullchain.pem certs/` и `cp /etc/letsencrypt/live/ваш-домен.ru/privkey.pem certs/`.

Завершающим шагом является запуск NGINX командой `docker compose up -d nginx`. Теперь приложение доступно по адресу `https://ваш-домен.ru`, а наличие зелёного замка в адресной строке браузера подтверждает валидность сертификата и безопасность соединения.

2.8 Развёртывание на сервере

Для успешного развёртывания приложения сервер должен соответствовать следующим требованиям: операционная система Ubuntu 22.04 или 24.04, установленные Docker и Docker Compose, открытые порты 80 и 443 для входящего трафика, а также домен, направленный на IP-адрес сервера с помощью А-записи.

Процесс развёртывания начинается с клонирования репозитория командой `git clone https://github.com/ваш-username/note-app.git` и перехода в папку проекта `cd note-app`. Далее создаётся файл с переменными окружения `.env`, в котором указываются домен, электронная почта для уведомлений Let's Encrypt, а также учётные данные для подключения к базе данных PostgreSQL[5]. После

настройки окружения все контейнеры запускаются одной командой `docker compose up -d`. Приложение становится доступно по адресу <https://ваш-домен.ru>.

2.9 Тестирование приложения

В ходе функционального тестирования были проверены основные сценарии работы приложения. При вводе заголовка и текста заметки с последующим нажатием кнопки «Сохранить» новая заметка успешно добавляется и отображается в общем списке (рисунок 1). При нажатии кнопки «Удалить» заметка исчезает из списка, что подтверждает корректную работу операции удаления.

Кроме того, была проведена проверка корректности настройки HTTPS-соединения. При переходе по адресу <https://jamikkk.work.gd> веб-приложение успешно загружается, а в адресной строке браузера отображается зелёный замок, подтверждающий использование защищённого соединения с валидным SSL-сертификатом Let's Encrypt (рисунок 2)(рисунок 3).

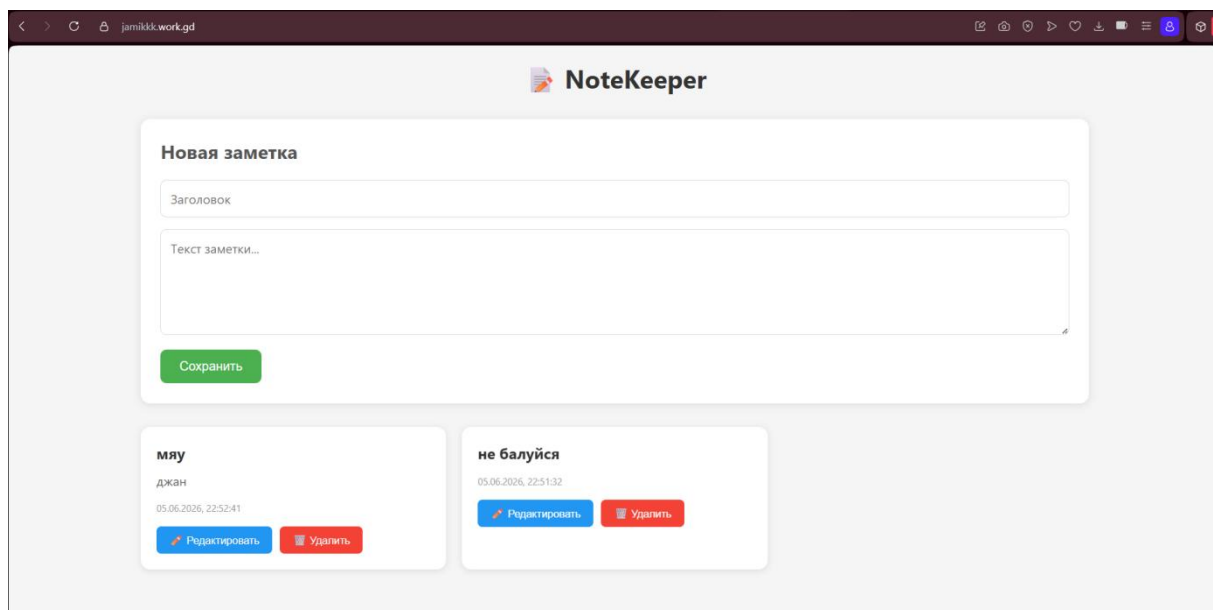


Рисунок 1. Главная страница приложения

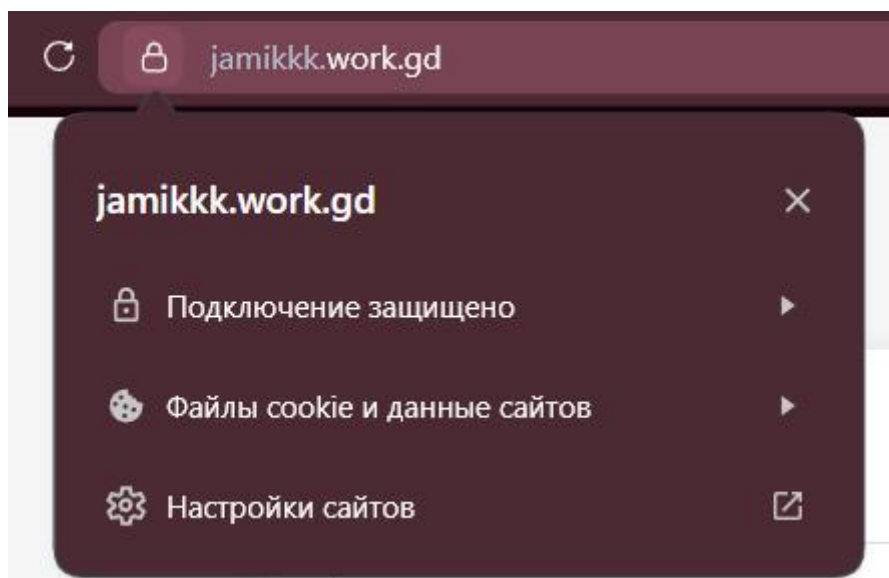


Рисунок 2. Защищённое подключение

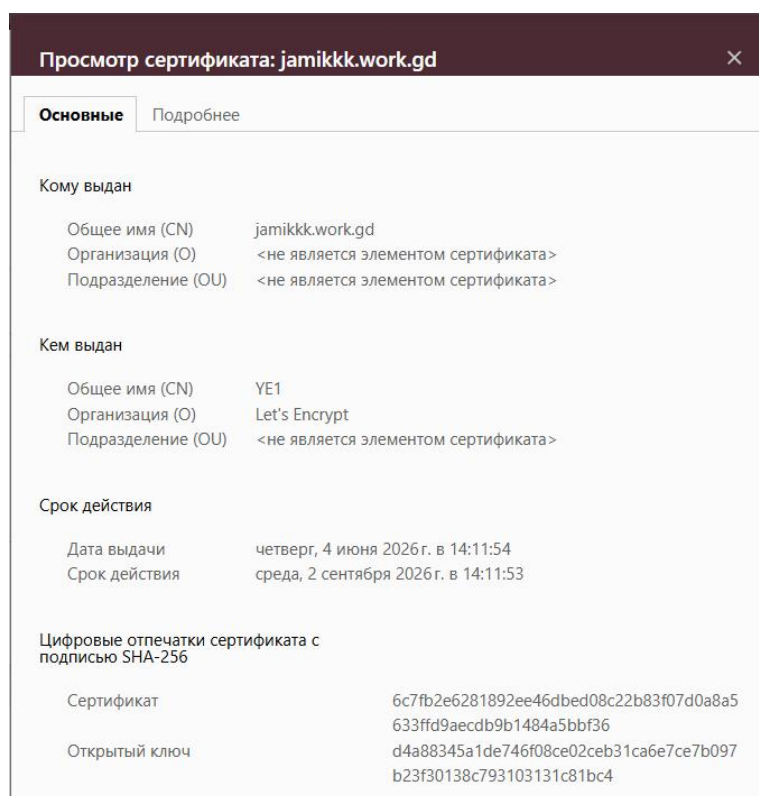


Рисунок 3. Сведения о сертификате

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсовой работы была достигнута поставленная цель — разработано веб-приложение для управления заметками, выполнена его контейнеризация с использованием Docker, настроен reverse-proxy на базе NGINX и организовано автоматическое получение SSL-сертификата Let's Encrypt.

В теоретической части работы были изучены основные технологии, используемые в проекте. Рассмотрена платформа контейнеризации Docker, позволяющая упаковывать приложения со всеми зависимостями в изолированные контейнеры. Изучен инструмент оркестрации Docker Compose, обеспечивающий декларативное описание и запуск многоконтейнерных приложений одной командой. Проанализирован веб-сервер NGINX, способный выполнять функции reverse-proxy и обеспечивать SSL-терминацию. Описана система Let's Encrypt — бесплатный автоматизированный центр сертификации, использующий протокол ACME для выдачи и обновления сертификатов. Также был проведён сравнительный анализ альтернативных технологий для каждого компонента системы, на основе которого обоснован выбор используемых решений.

В практической части работы были выполнены все поставленные задачи. Разработано веб-приложение «NoteKeeper»: серверная часть реализована на Node.js с использованием фреймворка Express.js и предоставляет REST API для работы с заметками; клиентская часть выполнена на нативном JavaScript с адаптивным веб-интерфейсом. Созданы Dockerfile для контейнеризации бэкенда и фронтенда, а также файл docker-compose.yml для оркестрации всех сервисов. Настроен reverse-proxy на базе NGINX с автоматическим перенаправлением HTTP-трафика

на HTTPS. С помощью инструмента Certbot получен SSL-сертификат Let's Encrypt для домена jamikkk.work.gd. Приложение успешно развёрнуто на VPS-сервере и доступно по защищённому протоколу HTTPS, что подтверждается наличием зелёного замка в адресной строке браузера. Проведено функциональное тестирование, подтвердившее корректную работу всех операций с заметками. Составлена документация по запуску проекта в формате README.md, оформленная по образцу Docker Hub.

Таким образом, все задачи, поставленные в начале работы, выполнены в полном объёме. Полученные в ходе работы навыки могут быть применены при развёртывании любых веб-приложений в production-среде с использованием современных DevOps-практик контейнеризации и автоматической настройки безопасности. Разработанные конфигурационные файлы и документация могут служить шаблоном для последующих проектов, требующих контейнеризации и поддержки HTTPS.

СПИСОК ЛИТЕРАТУРЫ

1. Docker Documentation. — Текст : электронный // Docker Documentation : [сайт]. — URL: <https://docs.docker.com> (дата обращения: 06.06.2026).
2. nginx. — Текст : электронный // NGINX Official Website : [сайт]. — URL: <https://nginx.org/en/> (дата обращения: 06.06.2026).
3. Let's Encrypt Official Documentation. — Текст : электронный // Let's Encrypt : [сайт]. — URL: <https://letsencrypt.org/docs/> (дата обращения: 06.06.2026).
4. Automatic Certificate Management Environment (ACME). — Текст : электронный // datatracker : [сайт]. — URL: <https://datatracker.ietf.org/doc/html/rfc8555> (дата обращения: 06.06.2026).
5. PostgreSQL Official Documentation. — Текст : электронный // postgresql : [сайт]. — URL: <https://www.postgresql.org/docs/> (дата обращения: 06.06.2026).